

AI Logic, Routine Flow and Task Scheduling – Report

Introduction

In most campuses, maintenance complaints are still handled through phone calls, WhatsApp groups or paper forms. Students don't know **who** to contact, technicians don't know **which job is truly urgent**, and admins struggle to get a clear picture of what is happening across all blocks and hostels.

Our smart maintenance portal tries to fix that by offering a single online portal where students can raise complaints, admins can monitor them, and technicians receive a clearly ordered task list. The special part of our system is that it does not treat all complaints equally: we use a small machine learning (ML) model to **predict the priority** of each complaint and drive the whole scheduling logic around that.

Below we describe, in simple language, how our AI logic works, how a typical day flows through the system, and how we schedule tasks for technicians.

1. AI Logic – Priority Prediction Engine

The core question we ask is:

“Given a new complaint, **how urgent** is it compared to others?”

Instead of letting students randomly mark things as “urgent”, we train a ML model that looks at multiple factors and predicts one of four priority levels:

- **P1 – Critical**
- **P2 – High**
- **P3 – Medium**
- **P4 – Low**

1.1 What information goes into the model?

For every complaint, we collect a set of features that we found to be important in real campus situations:

- **Problem category** – electrical, plumbing, civil, IT, etc.
- **Sub-category** – for example: “lift not working”, “fan not working”, “water leakage”.
- **Location** – hostel / block / lab / floor number.
- **Impact level selected by student** – low / medium / high / very high.
- **History of similar issues** – whether the same location had similar complaints recently.
- **Time of complaint** – whether it was raised during class hours, at night, etc.

Internally, we convert these into numbers (through label or one-hot encoding) so that the model can process them.

1.2 Model training

We treat this as a **supervised classification** problem. We prepare a dataset of past or sample complaints and manually tag them with correct priorities (P1–P4). For example:

- “Lift stuck with students inside” → P1
- “No water in hostel during morning hours” → P1
- “Broken classroom window but class is not using that room” → P3
- “Request to change tube light colour” → P4

The training pipeline is:

1. Clean the data and handle missing values.
2. Encode the categorical features.
3. Split the dataset into training and testing sets.
4. Train a tree-based model (Random Forest / Gradient Boosting style) which works well for mixed text-like features and gives decent accuracy even with moderate data.

After training, the model can take the features of a **new** complaint and output a priority label and, optionally, a probability score.

1.3 How prediction is used in the app

When a student presses **Submit** on the complaint form:

1. The backend collects all the relevant features from the form.
2. It calls our Python prediction script (the advanced version) and sends those features.
3. The script loads the trained model, runs the prediction, and returns a priority like **P2**.
4. The backend saves this priority along with the complaint in the database.
5. On the UI, the complaint gets a color-coded tag:
 - P1 – red “Critical”
 - P2 – orange “High”
 - P3 – yellow “Medium”
 - P4 – green “Low”.

From that moment, this AI-generated priority drives how the complaint appears in dashboards and how it is scheduled.

1.4 Learning from feedback

The system is not frozen after training. If an admin later changes a priority (for example, changes a P3 to P1 because it turned out to be serious), or a technician marks a task as “not urgent”, we treat that as feedback. These corrected labels can be added to the next training dataset so that the model gradually aligns better with real-world behaviour on that campus.

2. Routine Flow – How a Typical Day Looks

Three types of users interact with the system: **students, technicians, and admins**. The AI logic runs silently in the background, but the routine flow is designed to feel natural for each role.

2.1 Student journey

1. The student logs into **[Project Name]** using their USN and password stored in the **students** table.
2. They open the “Raise Complaint” page and fill in details:
 - problem category and sub-category
 - location (building, room, floor)
 - a short description
 - impact level and optional photo.
3. On submit, their complaint is sent to the backend, the AI model predicts priority, and the complaint is stored with status **OPEN**.
4. Immediately, the student sees a ticket ID, the predicted priority badge, and the current status.
5. Later, they can log in again to check the live status: **OPEN** → **IN-PROGRESS** → **RESOLVED**, and leave feedback after resolution.

From the student’s point of view, this feels like a simple help-desk portal, but internally it is being smart about urgency and assignment.

2.2 Technician journey

Each technician has a small profile with a **specialization** (e.g., “Lifts & Electrical” or “Plumbing & Civil”).

1. The technician logs in and is shown their **personal task queue**.
2. The queue lists only complaints related to their specialization, sorted by:
 - predicted priority (P1 at the top), and
 - within the same priority, by oldest first.
3. For each task, they see:
 - brief description and location

- priority tag
 - status and time since it was raised.
4. When they start working, they update the status to **IN-PROGRESS**.
 5. When the work is completed, they switch it to **RESOLVED** and add a short note (e.g., “Replaced MCB, tested OK”).

This makes their routine more structured: they do not need to guess what to do next; the system recommends it.

2.3 Admin journey

The admin has an overview of **the whole campus**.

1. On login, they see statistics such as:
 - total open complaints
 - distribution by priority
 - average resolution time per category / technician.
2. They can filter complaints by building, date or category, and view them on a map (if enabled).
3. Admins can:
 - override wrong priorities
 - assign or re-assign complaints to technicians
 - create routine maintenance templates (for example, “Weekly lift inspection” or “Monthly water tank cleaning”).

Thus, admins move from firefighting mode to a more controlled, data-driven view of campus maintenance.

3. Task Scheduling Logic

Task scheduling is where AI predictions are combined with simple rules to decide **who does what and when**.

3.1 Ordering complaints by urgency

For each complaint, we consider:

- the **priority** from the ML model (P1–P4)
- how **old** the complaint is (time since creation)
- whether there are **multiple complaints from the same location** recently.

In the database, complaints for a particular technician are typically fetched using a query like:

```
SELECT *  
FROM complaints  
WHERE status = 'OPEN'  
  AND category IN (technician_specialization)  
ORDER BY priority_level ASC, created_at ASC;
```

Here, **priority_level** is ordered so that $P1 < P2 < P3 < P4$, meaning P1 appears first. Within the same priority, older complaints come before newer ones, so nothing gets stuck forever.

Internally, you can think of an **urgency score** being formed by combining the priority, age, and repeated-location factor. We don't force technicians with formulas on the UI; we just show them the resulting ordered list.

3.2 Assigning complaints to technicians

When a new complaint is created, we immediately decide which technician should handle it:

1. From the complaint category, we identify the required specialization.
2. We look at all technicians with that specialization.
3. We calculate how many active tasks each technician already has.
4. The complaint is assigned to the technician with the **lightest current load** (and optionally nearest location), and their ID is stored in the complaint row.

This simple load-balancing prevents one technician from getting flooded while others are relatively free.

3.3 Routine / preventive tasks

Apart from reactive complaints from students, the system also supports **routine tasks**, which are essential for preventing failures.

1. Admins create “routine task templates” with:
 - task name
 - location and category
 - frequency (daily/weekly/monthly)
 - default priority and preferred time.
2. A small scheduler (cron job / scheduled function) runs in the backend. According to the frequency, it auto-creates complaints for today’s routine tasks, setting a flag `is_routine = true`.
3. These routine complaints then go through the **same assignment and ordering logic** as normal complaints, so technicians see them in their queue along with student issues.

As a result, technicians don’t have to remember all routine checks manually; the system gently pushes them at the right time.

Conclusion

In summary, it combines a familiar complaint portal UI with an ML-based priority engine and a simple but effective scheduling logic. Students get transparency, technicians get a clear to-do list, and admins gain data-driven control over campus maintenance.